simple: the more applications you have, the more traffic you can handle, everything else being equal.

However, scaling applications that use `IHostedService` to run background tasks might *not* be as simple. For example, imagine your application includes the `BackgroundService` that we created in section 34.1.2, which saves exchange rates to the database every five minutes. When you're running a single instance of your app, the task runs every five minutes as expected.

But what happens if you scale your application and run 10 instances of it? Every one of those applications will be running the `BackgroundService`, and they'll all be updating every five minutes from the time each instance started!

One option would be to move the `BackgroundService` to a separate worker service app. You could then continue to scale your ASP.NET Core application to handle the traffic as required but deploy a single instance of the worker service. As only a single instance of the `BackgroundService` would be running, the exchange rates would be updated on the correct schedule again.

TIP Differing scaling requirements, as in this example, are one of the best reasons for splitting bigger apps into smaller microservices. Breaking up an app like this has a maintenance overhead, however, so think about the tradeoffs if you take this route. For more on this tradeoff, I recommend *Microservices in .NET Core*, 2nd ed., by Christian Horsdal Gammelgaard (Manning, 2021).

However, if you take this route, you add a hard limitation that you can have only a single instance of your worker service. If you need to run more